PROGRAM

LEX

```
%{
#include "y.tab.h"
%}
%%
[0-9] {return DIGIT;}
[a-zA-Z] {return ALPHA;}
\n {return 0;}
. {return yytext[0];}
%%
```

YACC

```
%{
#include<stdio.h>
#include<stdlib.h>
int yylex(void);
int yyerror(const char *s);
%}
%token DIGIT ALPHA
%%
var : ALPHA
    | var ALPHA
    | var DIGIT
    ;
%%
int main()
{
printf("Enter the variable name:");
yyparse();
printf("Valid variable");
return 0;
}
int yyerror(const char *s)
{
printf("invalid variable");
exit(1);
return 0;
}
```

OUTPUT

```
Enter the variable name:hello
Valid variable
```

PROGRAM

LEX

```
%{
#include "y.tab.h"
#include <stdio.h>
#include <stdlib.h>
%}
%%
[0-9]+ {yylval.ival=atoi(yytext); return NUMBER;}
\n {return '\n';}
. {return yytext[0];}
%%
```

YACC

```
%{
#include<stdio.h>
#include<stdlib.h>
int yylex(void);
void yyerror();
%}
%union {int ival;}
%token<ival>NUMBER
%type<ival>exp
%left '+"-'
%left '*"/'
%%
st:exp '\n' {printf("Answer=%d\n",$1);};
exp:exp'+'exp {$$=$1+$3;}
   |exp'-'exp {$$=$1-$3;}
   |exp'*'exp {$$=$1*$3;}
   |exp'/'exp {$$=$1/$3;}
   |'('exp')' {$$=$2;}
   |NUMBER {$$=$1;}
   ;
%%
void main(){
printf("Enter the expression:");
yyparse();
}
int yywrap(){
return 1;
}
void yyerror(){
printf("Error");
}
```

OUTPUT

```
Enter the expression:(3+2)*2/2
Answer=5
```

PROGRAM

```
#include<stdio.h>
#include<math.h>
#include<string.h>
#include<ctype.h>
#include<stdlib.h>
int n,m=0,p,i=0,j=0;
char a[10][10],f[10];
void first(char c);
void follow(char c);
int main()
{
	int i,z;
	char c,ch;
	printf("Enter the No.of productions:\n");
	scanf("%d",&n);
	printf("Enter the productions:\n");
	for(i=0;i<n;i++)
		scanf("%s%c",a[i],&ch);
	do{
		m=0;
		printf("Enter the elements whose first and follow is to be found:");
		scanf("%c",&c);
		first(c);
		printf("First(%c)={",c);
		for(i=0;i<m;i++)
			printf("%c",f[i]);
		printf("}\n");
		strcpy(f," ");
		m=0;
		follow(c);
		printf("Follow(%c)={",c);
		for(i=0;i<m;i++)
			printf("%c",f[i]);
		printf("}\n");
		printf("Continue (0/1)?");
		scanf("%d%c",&z,&ch);
	}while(z==1);
	return(0);
}
void first(char c)
{
	int k;
	if(!isupper(c))
		f[m++]=c;
	for(k=0;k<n;k++)
	{
		if(a[k][0]==c)
		{
			if(a[k][2]=='$')
				follow(a[k][0]);
```

```
                else if(islower(a[k][2]))
                        f[m++]=a[k][2];
                else
                        first(a[k][2]);
                }
        }
}
void follow(char c)
{
        if(a[0][0]==c)
                f[m++]='$';
        for(i=0;i<n;i++)
        {
                for(j=2;j<strlen(a[i]);j++)
                        {
                        if(a[i][j]==c)
                        {
                                if(a[i][j+1]!='\0')
                                        first(a[i][j+1]);
                                if(a[i][j+1]=='\0'&&c!=a[i][0])
                                                follow(a[i][0]);
                                                }
                                        }
                                }
                        }
```

OUTPUT

```
Enter the No.of productions:
5
Enter the productions:
S=AbCd
A=Cf
A=a
C=gE
E=h
Enter the elements whose first and follow is to be found:S
First(S)={ga}
Follow(S)={$}
Continue (0/1)?1
Enter the elements whose first and follow is to be found:A
First(A)={ga}
Follow(A)={b}
Continue (0/1)?1
Enter the elements whose first and follow is to be found:C
First(C)={g}
Follow(C)={df}
Continue (0/1)?1
Enter the elements whose first and follow is to be found:E
First(E)={h}
```

Follow(E)={df}
Continue (0/1)?0

PROGRAM

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>
char in[100];
int i=0,err=0;
void E(),Ep(),T(),Tp(),F();
int main()
{
	printf("Enter expression:");
	fgets(in,sizeof(in),stdin);
	in[strcspn(in,"\n")]='\0';
	E();
	if(!err && in[i]=='\0')
		printf("Accepted\n");
	else
		printf("Rejected\n");
}

void E()
{
	T();
	Ep();
}
void Ep()
{
	if(in[i]=='+')
	{
		i++;
		T();
		Ep();
	}
}
void T()
{
	F();
	Tp();
}
void Tp()
{
	if(in[i]=='*')
	{
		i++;
		F();
		Tp();
	}
}
void F()
{
	if(in[i]=='(')
	{
```

```
            i++;
            E();
            if(in[i]==')')i++;
            else err=1;
      }
      else if(isalpha(in[i]))
      {
            i++;
            while(isalnum(in[i])||in[i]=='_')
                  i++;
      }
      else
            err=1;

}
```

OUTPUT

```
Enter expression:a+b*c
Accepted

Enter expression:a+*c
Rejected
```

PROGRAM

```
#include<stdio.h>
#include<string.h>
char op[2],arg1[5],arg2[5],result[5];
void main()
{
	FILE *fp1,*fp2;
	fp1=fopen("input.txt","r");
	fp2=fopen("output.txt","w");
	while(!feof(fp1))
	{
		fscanf(fp1,"%s%s%s%s",op,arg1,arg2,result);
		if(strcmp(op,"+")==0)
		{
			fprintf(fp2,"\nMOV R0,%s",arg1);
			fprintf(fp2,"\nADD R0,%s",arg2);
			fprintf(fp2,"\nMOV %s,R0",result);
		}
		if(strcmp(op,"*")==0)
		{
			fprintf(fp2,"\nMOV R0,%s",arg1);
			 fprintf(fp2,"\nMUL R0,%s",arg2);
			fprintf(fp2,"\nMOV %s,R0",result);
		}
		if (strcmp(op,"-")==0)
			{
			fprintf(fp2,"\nMOV R0,%s",arg1);
			 fprintf(fp2,"\nSUB R0,%s",arg2);
			fprintf(fp2,"\nMOV %s,R0",result);
		}
		if(strcmp(op,"/")==0)
			{
			fprintf(fp2,"\nMOV R0,%s",arg1);
			 fprintf(fp2,"\nDIV R0,%s",arg2);
			fprintf(fp2,"\nMOV %s,R0",result);
		}
		if(strcmp(op,"=")==0)
			{
			 fprintf(fp2,"\nMOV R0,%s",arg1);
			fprintf(fp2,"\nMOV %s,R0",result);
		}
	}
	fclose(fp1);
	fclose(fp2);
}
```

input.txt

```
+ a b t1
- c d t2
* t1 t2 t3
/ t3 t1 t4 ?x
```

OUTPUT

```
MOV R0,a
ADD R0,b
MOV t1,R0
MOV R0,c
SUB R0,d
MOV t2,R0
MOV R0,t1
MUL R0,t2
MOV t3,R0
MOV R0,t3
DIV R0,t1
MOV t4,R0
```

PROGRAM

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<ctype.h>
char in[50],st[50];
int ip=0,sp=0,n;
void reduce(){
	int f=1;
	while (f){
		f=0;
		char t;
		if(st[sp-1]=='a'||st[sp-1]=='b'||st[sp-1]=='c')
		{
			t=st[sp-1];
			st[sp-1]='E';
			printf("\n$%s\t\t%s\t\tE->%c",st,in+ip,t);
			f=1;
		}
		if(sp>=3 && st[sp-3]=='E' && strchr("+-*/",st[sp-2]) && st[sp-1]=='E')
		{
			char op=st[sp-2];
			st[sp-3]='E';
			st[sp-2]=st[sp-1]='\0';
			sp-=2;
			printf("\n$%s\t\t%s\t\tE->E%cE",st,in+ip,op);
			f=1;
		}
	}
}

int main()
{
	printf("\nSHIFT REDUCE PARSER\n\nGrammar:\nE->E+E\nE->E*E\nE->E/E\nE->a/b/c\n");
	printf("\nEnter input: ");
	scanf("%s",in);
	n=strlen(in);
	in[n++]='$';
	in[n]='\0';
	printf("\nStack\t\tInput\t\tAction\n$\t\t%s\t\t--",in);
	while (1)
	{
		if(!(isalpha(in[ip])||strchr("*/+$",in[ip])))
		{
			printf("\n$%s\t\t%s\t\tREJECT",st,in+ip);
			return 0;
		}
		st[sp++]=in[ip];
		st[sp]='\0';
		printf("\n$%s\t\t%s\t\tshift %c",st,in+ ++ip,st[sp-1]);
		reduce();
```

```
            if(!strcmp(st,"E") && in[ip]=='$'){
                printf("\n$%s\t\t%s\t\tACCEPT",st,in+ip);
                break;
            }
            if(ip>=n && strcmp(st,"E"))
            {
                printf("\n$%s\t\t%s\t\tREJECT",st,in+ip);
                break;
            }
        }
}
```

OUTPUT

SHIFT REDUCE PARSER

Grammar:
E->E+E
E->E*E
E->E/E
E->a/b/c

Enter input: a+b*c

Stack	Input	Action
$	a+b*c$	--
$a	+b*c$	shift a
$E	+b*c$	E->a
$E+	b*c$	shift +
$E+b	*c$	shift b
$E+E	*c$	E->b
$E	*c$	E->E+E
$E*	c$	shift *
$E*c	$	shift c
$E*E	$	E->c
$E	$	E->E*E
$E	$	ACCEPT

PROGRAM

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>
#include<stdlib.h>
struct expr{
     char op[2],op1[5],op2[5],res[5];
     int flag;
} arr[10];
int n;
void change(int p,char *res){
  for(int i=p+1;i<n;i++){
    if(!strcmp(arr[p].res,arr[i].op1))strcpy(arr[i].op1,res);
    if(!strcmp(arr[p].res,arr[i].op2))strcpy(arr[i].op2,res);
 }
}
void constant() {
  for(int i=0;i<n;i++) {
   if((isdigit(arr[i].op1[0]) && isdigit(arr[i].op2[0])) || !strcmp(arr[i].op,"=")) {
    int op1=atoi(arr[i].op1),op2=atoi(arr[i].op2),res;
    switch(arr[i].op[0]) {
      case'+':res=op1+op2;break;
      case'-':res=op1-op2;break;
      case'*':res=op1*op2;break;
      case'/':res=op1/op2;break;
      case'=':res=op1;break;
}
char res1[5];sprintf(res1,"%d",res);
arr[i].flag=1;
change(i,res1);
   }
 }
}
int main() {
   printf("Enter number of expressions:");
   scanf("%d",&n);
   printf("Enter expressions as input:\n");
   for(int i=0;i<n;i++) {
         scanf("%s%s%s%s",arr[i].op,arr[i].op1,arr[i].op2,arr[i].res);
         arr[i].flag=0;
       }
   constant();
   printf("\nOptimized code:\n");
   for(int i=0;i<n;i++)
   if(!arr[i].flag) printf("%s%s%s%s\n",arr[i].op,arr[i].op1,arr[i].op2,arr[i].res);
}
```

OUTPUT

Enter number of expressions:4
Enter expressions as input:
= 10 0 a
= 20 0 b
+ a b c
+ c x d

Optimized code:
+30xd